



Desenvolvimento de Sistemas

Técnicas de teste: caixa-preta, caixa-branca; métodos de testes

Introdução

Utilizar técnicas de teste consistentes e aprovadas pelo mercado de TI (tecnologia da informação) tornará o desenvolvimento do projeto mais ágil e assertivo em seu resultado final. Atualmente, as técnicas são divididas em: funcionais e não funcionais.

Os testes funcionais validam as funções do sistema, ou seja, serão realizados testes nas funcionalidades descritas no documento de requisitos funcionais. Devem ser produzidos testes para contemplar todos os níveis. Geralmente medida em valores percentuais, a cobertura define a eficácia do plano de testes elaborado, tem como técnica conhecida a caixa-preta e baseia-se em experiências.

Um exemplo de teste funcional é em um determinado *software* de controle de estoque em que o operador do sistema deve emitir uma nota fiscal com diferentes produtos e, ao final da operação, o sistema tem como ação esperada a dedução dos itens vendidos da base de estoque.

Testes não funcionais verificarão: segurança, usabilidade, velocidade e demais características envolvidas no escopo do projeto, considerando as regras de negócio e as restrições envolvidas no projeto.

Para testes não funcionais, é possível citar como exemplo a seguinte situação: a empresa está para realizar a inserção de 50 novos operadores para trabalhar simultaneamente aos 20 que já interagem no sistema. Ao total, haverá 70 operadores que emitirão nota fiscal ao mesmo tempo. Dessa forma, será avaliada a *performance* do sistema após a escalabilidade realizada.

Outro exemplo de teste não funcional é a análise de comportamento do sistema ao sofrer uma tentativa de invasão à base de dados por meio de uma técnica de acesso não permitido. Nesse caso, um teste de segurança estará sendo realizado. Ainda, é possível testar a usabilidade do sistema ao avaliar como pessoas de diferentes faixas etárias se portarão ao utilizar o sistema.

Estas são algumas técnicas utilizadas no mercado:



Teste de caixa-branca: consiste na utilização do código-fonte do projeto e também é conhecido como teste estrutural.



Teste de caixa-preta: considera as funcionalidades do sistema, deixando de lado o código-fonte e testando apenas as funções desenvolvidas.



Teste de caixa-cinza: consiste na junção dos testes de caixa-branca e caixa-preta e utiliza a engenharia reversa, sendo possível, assim, analisar erros e problemas gerados no desenvolvimento do projeto.

Nesta unidade curricular, serão abordados com mais ênfase o teste de caixa-branca e o teste de caixa-preta.

Caixa-preta

Essa técnica considera as entradas e saídas esperadas para o **software**, sem levar em consideração a estrutura do código-fonte. Dessa forma, é avaliado se a estrutura está operando da forma especificada na documentação, seguindo os requisitos funcionais, por isso também é conhecida como teste de comportamento. A técnica da caixa-preta não permite a verificação do código desenvolvido para validar se está realizando uma função de forma otimizada ou não.

Geralmente tem um ambiente de teste seguindo as premissas estipuladas em casos de uso, *user stories* e especificações do sistema. Os casos de testes encontrarão possíveis falhas entre o que foi especificado e o que realmente está acontecendo. A cobertura é parametrizada,

considerando os itens testados na base e a técnica aplicada.



Para mais informações sobre o nível de cobertura, busque em seu navegador por “padrão internacional ISO/IEC/IEEE 29119-4”.

Métodos de teste

Agora, você saberá mais a respeito dos métodos de teste.

Particionamento de equivalências

Imagine o seguinte caso: uma nova empresa de tintas está desenvolvendo um *software* para validar o preenchimento de seus tanques de tintas e definiu que um tanque deve ao menos ter 10 l de tintas e no máximo 50 l. Com isso, ao cadastrar o preenchimento do tanque, o campo de litros poderá aceitar valores entre 10 e 50.

Para facilitar o entendimento, analise a linha de valores que podem ou não ser inseridos no momento de realizar a tarefa.

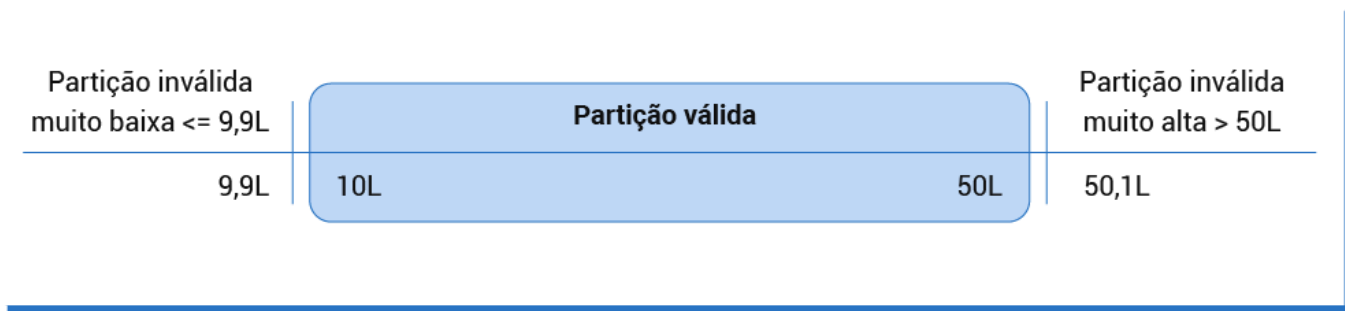


Figura 1 – Método de teste particionamento de equivalência

Fonte: Senac EAD (2023)

Ao analisar a imagem, percebem-se três partições: partição inválida muito baixa, partição válida e partição inválida muito alta. Os valores para a partição muito baixa estão abaixo da quantidade de litros que limita a disponibilidade de reserva, já a partição inválida muito alta limitará os valores acima da quantidade de litros que limita a disponibilidade. A partição válida contém os valores que são aceitos para definir cada quantidade de litros que permitem o preenchimento do tanque.

É o processo de divisão de dados, de modo que todos os membros pertencentes a uma partição serão processados do mesmo modo. Além disso, consiste na criação de partições para valores válidos e inválidos. Dessa forma, cria-se uma divisão para os valores que serão aceitos (válidos) e os que não serão aceitos (inválidos). Muitas vezes é criado de forma “inconsciente”.

A cobertura é feita por meio do seguinte cálculo:

$$COBERTURA = (PARTIÇÕES TESTADAS \div TOTAL DE PARTIÇÕES) * 100\%$$

Agora, imaginando os casos de teste que devem ser criados para cobrir essa situação, quantos seriam necessários para atender a todas as partições?

Devem ser criados três casos: um caso de teste com uma quantidade inferior ao mínimo de 10 l, um caso de teste com uma quantidade entre 10 l e 50 l e um caso de teste com uma quantidade superior a 50 l.

As partições inválidas devem ser testadas separadamente, ou seja, em partições diferentes, para evitar que uma falha possa mascarar as demais falhas que possam ocorrer. Desta forma um erro impediria os demais de serem detectados.

Análise de valor-limite

É uma ampliação do particionamento de equivalência e constitui-se de dados numéricos ou sequenciais, em que os valores mínimo e máximo são os valores-limite para uma partição. Pode ser aplicada em qualquer nível de teste e tem uma probabilidade maior de encontrar falhas em relação à partição de equivalência. A cobertura é definida da seguinte maneira:

$$COBERTURA = (LIMITES TESTADOS \div TOTAL DE LIMITES) * 100\%$$

Agora, pense no exemplo anterior, em que o limite no campo de litros foi implementado:

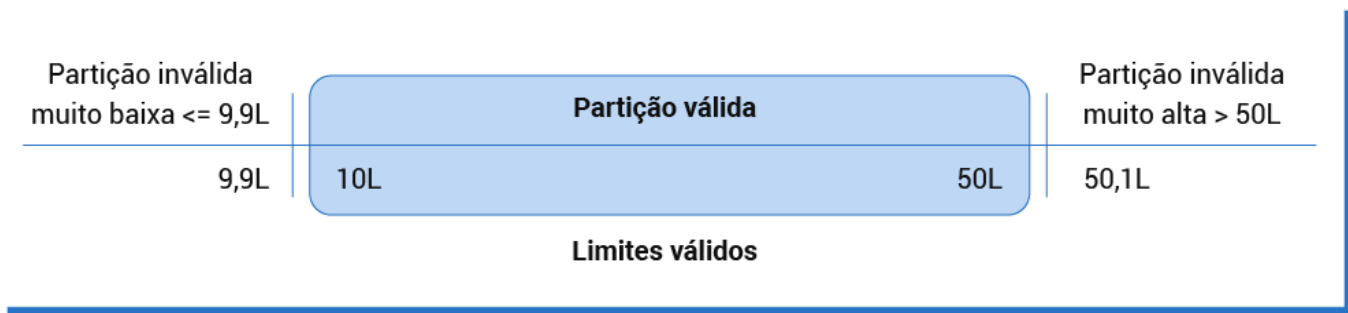


Figura 2 – Método de teste análise de valor-limite

Fonte: Senac EAD (2023)



Com base no conhecimento adquirido, você já sabe que valores abaixo de 10 l fazem parte da partição inválida muito baixa, que a partição válida contém os valores entre 10 l e 50 l e que a partição inválida muito alta tem valores acima de 50 l. Porém, agora está sendo feita apenas a análise dos valores-limite, portanto apenas 10 l e 50 l são os limites válidos para a partição válida. O limite da partição inválida muito baixa é somente 9,9 l e o limite de 50,1 l é o correspondente à partição inválida muito alta.

Agora, imaginando os casos de teste que devem ser criados para cobrir os limites dessa situação, quantos seriam necessários para atender a todas as partições?

Nessa situação, você precisa de quatro casos de teste, sendo um teste para cobrir o valor-limite da partição inválida muito baixa, nesse caso 9,9 l; um teste para cobrir o valor-limite mínimo válido da partição válida, nesse caso 10 l; um teste para cobrir o valor-limite máximo válido da partição válida, nesse caso 50 l; e um teste para cobrir o valor-limite da partição inválida muito alta, nesse caso 50,1 l.

Agora é hora praticar criando o código para esse projeto. Primeiro, você desenvolverá a base, criando uma variável para armazenar o nome que referenciará o tanque de tinta e exibirá os dados cadastrados ao final.

```
import java.util.Scanner;

public class TesteTanque {

    public static void main(String[] args) {
        String identificacao;
        double qtdTinta;
        Scanner entrada = new Scanner(System.in);

        System.out.println("Informe a identificação do tanque: ");
        identificacao = entrada.nextLine();
        System.out.println("Informe a quantidade de litros inserido no tanque: ");
        qtdTinta = entrada.nextDouble();

        System.out.println("O nome do tanque é: " + identificacao + " e a quantidade presente de litros é: " + qtdTinta);
    }
}
```

Agora, você realizará os testes de análise de valor-limite e verificará se está cobrindo os requisitos esperados para o projeto. Então, execute o código e analise o que acontece. Primeiro, informe um valor mínimo aceito de 10 l e, posteriormente, execute o código novamente e informe um valor máximo aceito de 50 l. Em ambos os casos deve ser exibida a mensagem de litros presentes no tanque e a identificação dele.

Então, você realizará os testes para os valores abaixo do mínimo aceito e acima do valor máximo aceito, portanto execute o código novamente. Primeiro, informe 9,9 l e veja o que acontece.

Execute outra vez o código e informe 50,1 l. Em ambos os casos deve ser exibida uma mensagem de valores inválidos informados.

Você percebeu que o código apresentou falhas, pois está faltando algo, não é mesmo? No teste, foram encontradas falhas do projeto; as estruturas de decisão não foram criadas para validar os valores que devem ser aceitos, ou seja, a cobertura do teste é de 50%. Nesse momento, você reportará, em um documento adequado, que essa função do sistema não está adequada ao que era esperado no documento de requisitos e deve ser refeita.

Portanto, refaça e adicione as validações que devem estar presentes.

```
import java.util.Scanner;

public class TesteTanque {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here
        String identificacao;
        Double qtdTinta;
        Scanner entrada = new Scanner(System.in);

        System.out.println("Informe a identificação do tanque: ");
        identificacao = entrada.nextLine();
        System.out.println("Informe a quantidade de litros inserido no tanque: ");
        qtdTinta = entrada.nextDouble();
        if(qtdTinta>=10 && qtdTinta<=50){
            System.out.println("O nome do tanque é: " + identificacao + " e a quantidade presente de litros é: " + qtdTinta);
        } else {
            System.out.println("A quantidade de tinta informada é inadequada para a capacidade do tanque");
        }
    }
}
```

}

Agora, você deve refazer os testes de análise de valor-limite e verificar se o projeto cobre os requisitos esperados.

Como era esperado, dessa vez os valores-limite inválidos, muito baixo e muito alto, não são aceitos. Já os limites válidos são aceitos, portanto o seu teste tem 100% de cobertura.

Tabela de decisão

A técnica de tabela de decisão testa combinações que resultarão em dados diferentes, sendo considerada interessante para cobrir regras de negócio complexas que um sistema tem. A tabela é composta de condições, que são as entradas, e ações (os resultados gerados na saída), que são as linhas. Para as colunas, há regras que indicarão quais condições têm as ações esperadas. O número de colunas é baseado no número de casos de testes.

As condições são expressas da seguinte forma:

“S”: especifica que a condição é atendida (em algumas literaturas, pode ser exibido como “V” ou “1”).
“N”: especifica que a condição não é atendida (em algumas literaturas, pode ser exibido como “F” ou “0”).
“-”: especifica que o valor não é importante para a condição (em algumas literaturas, pode ser exibido como “N/A”).

Tabela 1 – Condições

Fonte: Senac EAD (2023)

Para as ações, há as seguintes expressões:

“X”: especifica que a ação deve ocorrer (em algumas literaturas, pode ser exibido como “S”, “V” ou “1”).
Em branco: especifica que a ação não deve ocorrer (em algumas literaturas, pode ser exibido como “-”, “N”, “F” ou “0”).

Tabela 2 – Ações

Fonte: Senac EAD (2023)

Agora, pense na seguinte situação: uma *startup* de recursos humanos está desenvolvendo uma plataforma para facilitar a contratação de colaboradores e deseja disponibilizar para os clientes uma ferramenta que verifica se o candidato tem as qualificações esperadas para a vaga. Dessa forma, alguns requisitos devem ser expressos: ter formação esperada para a área, ter nível de experiência adequado, conforme especificação do cliente, e ter domínio no idioma solicitado.

Os candidatos que tiverem os requisitos necessários, ou seja, atenderem às condições, participarão da próxima etapa do processo seletivo. Os que não atenderem a um ou mais requisitos irão para o banco de talentos da empresa, sendo esta a ação esperada.

Veja a tabela de decisão:

CONDIÇÕES	Regra 1 (caso de teste 1)	Regra 2 (caso de teste 2)	Regra 3 (caso de teste 3)	Regra 4 (caso de teste 4)	Regra 5 (caso de teste 5)	Regra 6 (caso de teste 6)	Regra 7 (caso de teste 7)	Regra 8 (caso de teste 8)
Formação esperada para a área?	S	S	S	S	N	N	N	N
Nível de experiência adequado?	S	S	N	N	S	S	N	N
Domínio no idioma solicitado?	S	N	S	N	S	N	S	N
AÇÕES								
Participar da próxima etapa do processo seletivo	X							
Banco de talentos da empresa		X	X	X	X	X	X	X
LEGENDA: S = sim; N = não; X = ação a ser executada								

Tabela 3 – Tabela de decisão

Fonte: Senac EAD (2023)

Dessa forma, é possível verificar diferentes casos de teste e analisar o resultado de cada um, sendo, assim, uma técnica adequada para cobrir os requisitos mais complexos. No exemplo, apenas o primeiro caso de teste cobre por completo os requisitos e seguiria no processo seletivo; os demais iram para o banco de talentos.

Transição de estado

O método de transição de estados mostrará, por meio de um diagrama, os possíveis estados que o *software* pode ter, exibindo o modo como este alterna entre os estados, passando por entrada, saída e transição.

As transições são iniciadas com base em um evento, que, por exemplo, pode ser a inserção de dado em um campo. Com isso, uma mudança de estado pode gerar a execução de uma ação, que pode ser, por exemplo, a exibição de uma mensagem de confirmação na tela ou um erro em razão de inconsistências.

Há uma tabela de transição de estado que exibirá as transições válidas e inválidas entre estados. Os eventos também fazem parte da tabela. Já os diagramas mostram apenas transições válidas, deixando de fora as transições inválidas.

Pense em uma situação de abastecimento do tanque de tintas e elabore uma tabela de transição de estados. Há uma bomba de abastecimento que encherá o tanque ou ele poderá receber uma determinada quantidade de litros de tinta.

A tabela de testes de transição de estados ficará da seguinte forma:

Número do teste	Estado inicial	Ação	Estado final	Resultado esperado
1	Tanque vazio	Abastecer tanque	Tanque cheio	Tanque cheio
2	Vazio	Abastecimento com uma quantidade X	Tanque com valor X	Tanque com X litros de tinta
3	Abastecimento em andamento	Cancelar abastecimento	Vazio	Ao cancelar a ação, o tanque não realiza o abastecimento
4	Tanque cheio	Abastecimento com uma quantidade X de litros	Tanque com valor X de litros	Tanque já está cheio e não é possível abastecer
5	Tanque com valor X de tinta	Abastecimento com uma quantidade X de litros	Tanque com valor X de litros	Não é possível abastecer, pois ultrapassará a quantidade máxima do tanque
6	Falha do sistema	Cancelar abastecimento	Vazio	Em falha, o tanque não deve abastecer

Tabela 4 – Tabela de testes de transição de estados

Fonte: Senac EAD (2023)

A tabela de testes de transição de estados para a bomba de abastecimento mostra os diferentes estados que a bomba pode estar e as ações que o usuário pode realizar, levando a um estado final esperado. A tabela começa com o estado vazio da bomba e as ações de informar um valor específico para abastecimento ou escolher encher o tanque completamente. Quando o usuário escolhe informar um valor X, a bomba passa para o estado de abastecimento em andamento e começa a realizar o abastecimento. Se o usuário cancelar, a bomba volta ao estado vazio. Se a bomba estiver cheia, o usuário não poderá adicionar mais combustível, independentemente da ação escolhida. Em ambos os casos, o sistema informará ao usuário que o tanque já está cheio. No caso de falha do sistema, a bomba volta ao estado vazio.

Geralmente os testes são criados de forma a percorrer todos os estados, passar por cada transição ou cobrir uma sequência predefinida de transições ou transições inválidas.

Teste de caso de uso

O método de teste de caso de uso é baseado nos casos de uso. Desse modo, especificará o comportamento que uma funcionalidade terá ao interagir com um ou mais atores do sistema.

Os testes de caso de uso ajudam a garantir a qualidade e a confiabilidade do sistema, validando situações de uso e verificando se situações excepcionais têm o tratamento de falhas de sistema adequados. A medida de cobertura é expressa pela quantidade de comportamentos de casos de uso que serão testados dividida pelo total de comportamentos de caso de uso.

Caixa-branca

A técnica de teste de caixa-branca analisa a arquitetura do projeto, avaliando de modo interno a estrutura, sendo, assim, possível resolver questões de otimização do projeto, gerando um processamento mais adequado. Dessa forma, é possível criar testes que percorrerão mais trechos do código, evitando que ocorram redundâncias ou testes em duplicidade por meio de *scripts* criados para executar os testes de forma automática.

Ainda é possível realizar uma análise de cobertura de código. Dessa forma, é possível mensurar qual percentual do código é verificado e, com base nesse dado, criar testes complementares para validar o percentual faltante. A cobertura é medida com base nos itens testados de uma estrutura, de acordo com a técnica utilizada.

Por exemplo, ao criar um teste para cobrir todas as estruturas de decisão, os **ifs/elses** criados serão testados para verificar se o comportamento do sistema é o adequado ao passar por aquele trecho. Essa técnica tem um cálculo específico para verificar o percentual dos testes que obtiveram êxito ao serem executados.

Como se baseia na estrutura interna do objeto, as classes, as funções e os pacotes são verificados para validar a cobertura. Pode ser realizado em todos os níveis de teste, porém é mais utilizado para teste de componentes ou de integração de componentes.

Teste de cobertura de instruções

Verifica as instruções executáveis de um código. Tem como cálculo para percentual de cobertura a seguinte fórmula:

$$(\text{Instruções Executadas} / \text{Total de Instruções Existentes}) * 100$$

É a técnica de cobertura menos eficiente, pois verifica apenas se as instruções estão sendo executadas, desconsiderando outros tipos de cobertura, como situações condicionais. Em algumas literaturas, pode ser encontrada como cobertura de declaração, comando ou até mesmo sentença.

Agora, pense em um exemplo prático: em uma competição de salto em altura, um atleta tem três oportunidades para registrar a maior marca de seu salto. Você foi convidado a desenvolver um *software* para aferir a maior marca de um atleta, portanto construirá o seguinte código:

```
import java.util.Scanner;

public class Competicao {

    public static void main(String[] args) {
        // TODO code application logic here
        Scanner entrada = new Scanner(System.in);
        double salto1, salto2, salto3, maior;
        System.out.println("informe a altura do salto 1");
        salto1 = entrada.nextDouble();
        maior = salto1;
        System.out.println("Informe a altura do segundo salto");
        salto2 = entrada.nextDouble();
        System.out.println("Informe a altura do terceiro salto");
        salto3 = entrada.nextDouble();
        if (salto2 > maior) {
            maior = salto2;
            if (salto3 > maior) {
                maior = salto3;
            }
        } else {
            if (salto3 > maior) {
                maior = salto3;
            }
        }
        System.out.println("A maior marca é de: " + maior + " ms");
    }
}
```

Agora, pense no seguinte caso de teste para verificar a cobertura das instruções: o primeiro salto teve altura de 1,89 m, o segundo salto de 1,95 m e o terceiro foi anulado, portanto fica com 0,0 m. Dessa forma, teste seu código com esses valores para verificar quais instruções serão executadas. Para facilitar a análise, crie um fluxograma com as instruções do programa.

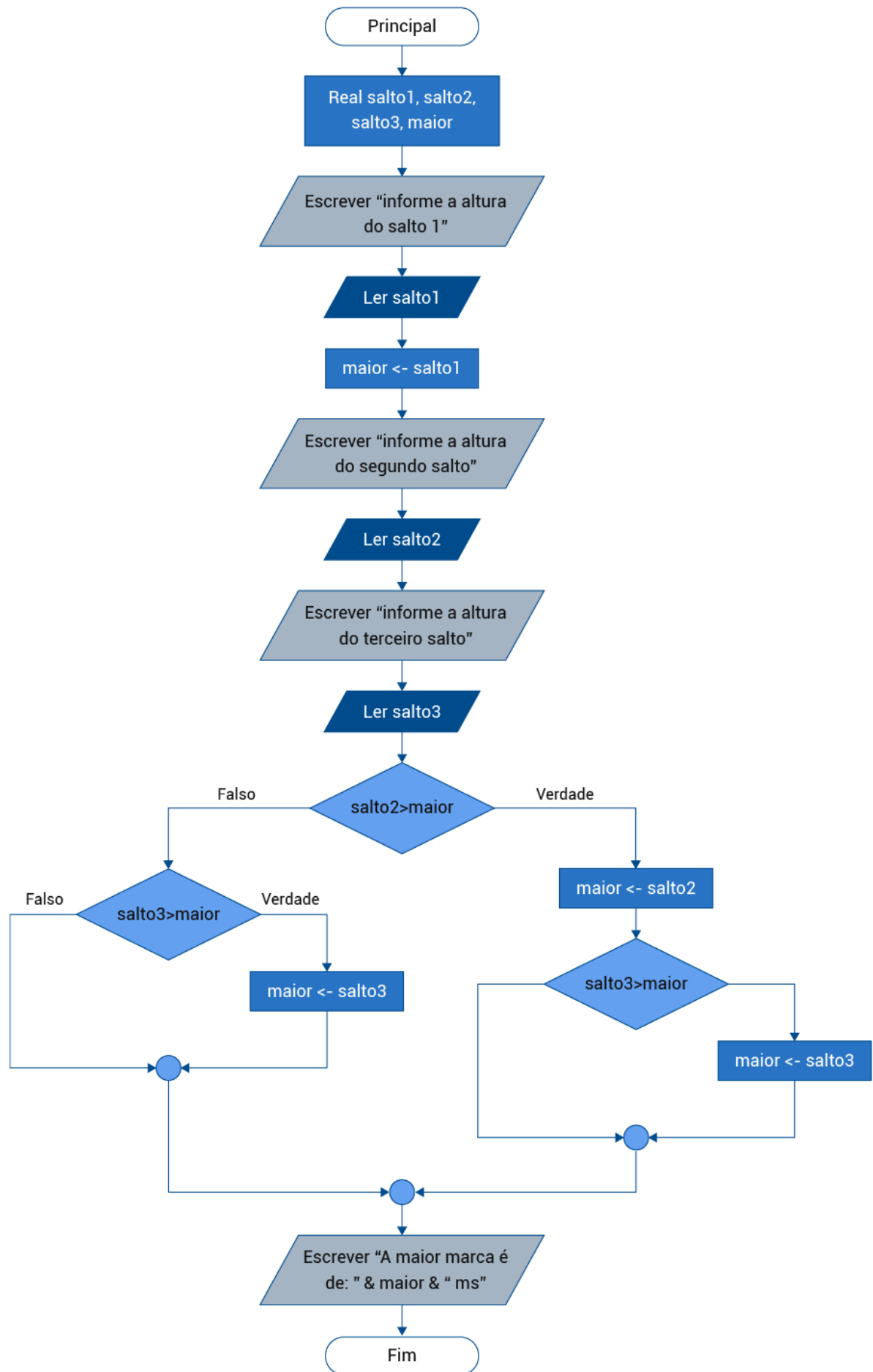


Figura 3 – Método de teste cobertura de comandos

Fonte: Senac EAD (2023)



Cada figura geométrica no fluxograma significa uma instrução, portanto há 15 (ficarão de fora os círculos rosas, que estão vazios, as setas e, em roxo, as elipses **Principal** e **Fim**).

Agora, analise as instruções. O programa realizará as declarações, as entradas/saídas na tela e as atribuições até chegar na primeira estrutura de decisão. Até a estrutura de decisão, oito instruções foram realizadas. A nona instrução a ser executada será a verificação de atendimento da condição. Com os dados desse caso de teste, o segundo salto é de 1,95 m, portanto maior que o valor que já está armazenado na variável maior. Logo, será executada também a instrução de atribuição de valor maior na variável maior, que é a décima instrução a ser testada.

A próxima instrução a ter sua cobertura verificada é a de análise do terceiro salto (não perca a conta ,11°), se ele é maior que o atual valor em maior. Como o terceiro salto foi anulado, a condição não é atendida e, assim, o programa segue para o 12° comando, que encerra a execução exibindo uma mensagem com a maior altura saltada. Considerando o cálculo de cobertura, foram executadas 12 instruções em um total de 15, portanto a cobertura desse teste é de 80%.

Teste de cobertura de decisão

Esse método de teste verifica as estruturas de decisões presentes no código, assim como as instruções que serão executadas conforme o resultado das condições. Os casos de testes devem ser criados de forma a validar situações condicionais. Por exemplo, se for uma estrutura **if**, deve-se criar um caso de teste para quando a condição é atendida e um para quando a condição não é atendida (verdadeiro ou falso, respectivamente). Em uma estrutura **switch case**, todos os casos devem ter um caso de teste que cobrirá o resultado, incluindo a resposta padrão.

A cobertura é medida por meio do seguinte cálculo:

$$(\text{Número de resultados de decisão executados} / \text{Total de decisões existentes}) * 100$$

Em algumas literaturas, utilizam-se termos como “ramificação” e “desvio”.

Agora, pense no seguinte exemplo: uma empresa está considerando subsidiar um auxílio para seus funcionários realizarem um curso de especialização em qualquer área, independentemente de ter ou não relação com seu cargo na empresa, no entanto o funcionário deve aceitar que o percentual restante seja descontado diretamente em folha de pagamento. O colaborador poderá, ainda, definir se, ao escolher realizar a especialização, receberá um outro subsidio de transporte, novamente descontado em folha.

Quantos e quais seriam os casos de uso necessários para cobrir todas as estruturas de decisão presentes no seguinte trecho de código?

```
import java.util.*;
import java.lang.Math;

public class JavaApplication {
    private static Scanner input = new Scanner(System.in);

    public static void main(String[] args) {
        String curso;
        String transporte;

        String curso;
        String transporte;

        System.out.println("Informe sim,se desejar realizar o curso de especialização ou não para recusar!");
        curso = input.nextLine();
        if (curso.equals("sim")) {
            System.out.println("Informe sim,se desejar ganhar o subsidio para transporte ou não para recusar!");
            transporte = input.nextLine();
            if (transporte.equals("sim")) {
                System.out.println("Você optou não receber o subsidio do transporte e fará a especialização");
            } else {
                System.out.println("Você optou por não receber o subsidio do transporte, mas fará a especialização");
            }
        } else {
            System.out.println("Você optou por não realizar a especialização");
        }
        System.out.println("Obrigado por participar");
    }
}
```

Use novamente um fluxograma para auxiliar na análise da execução.

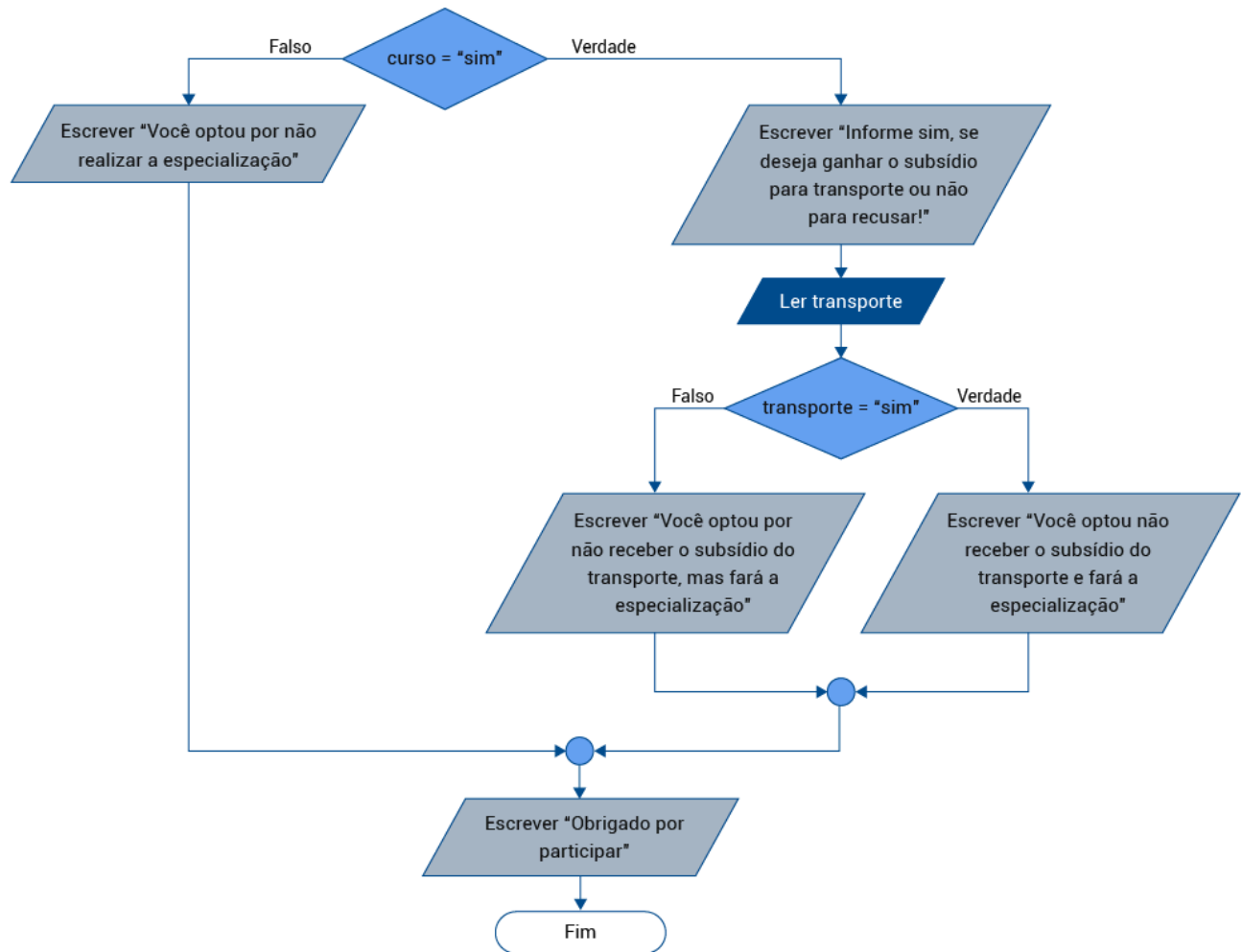


Figura 4 – Método de teste cobertura de decisão

Fonte: Senac EAD (2023)

O primeiro caso de teste seria definir a resposta para a participação no curso como “não”. Dessa forma, encerra a execução do programa, pois irá pelo caminho de decisão falso.

O segundo caso de teste seria definir a primeira resposta como “sim” e a segunda como “sim” novamente. Assim, passaria por todas as condições verdadeiras.

O terceiro caso de teste seria definir a primeira resposta como “sim” e a segunda como “não”. Dessa forma, percorreria pelo caminho da primeira decisão sendo verdadeiro e o caminho da segunda decisão sendo falso. Logo, seriam necessários três casos de teste para atingir a cobertura de 100% dos caminhos disponíveis conforme as estruturas de condição existentes.

Dessa forma, pode-se afirmar que cobrir 100% de um teste de cobertura de decisão cobrirá 100% de um teste de comando.

Desafio 1: Desafio de teste de tabela de decisão

Você é um desenvolvedor de *software* para uma empresa de seguros automotivos. Sua equipe está desenvolvendo um sistema para calcular o valor do seguro de carros, levando em consideração vários fatores, como idade do motorista, modelo do carro e ano do carro.

Seu desafio é criar uma tabela de decisão de teste que cubra os cenários mais importantes para o cálculo do valor do seguro. Para isso, considere as seguintes premissas:

- ◆ O modelo e o ano do carro também afetam o valor do seguro. Carros 0 km e de modelos acima de R\$ 60 mil terão um valor de 10% da tabela Fipe (tabela da Fundação Instituto de Pesquisas Econômicas).
- ◆ O valor do seguro é influenciado pela idade do motorista. Se o motorista tiver menos de 25 anos, o valor do seguro terá um acréscimo de 40%.
- ◆ Se o motorista tiver mais de 60 anos, o valor do seguro será automaticamente reduzido em 30%.

Desafio 2: Desafio de teste de cobertura de decisão

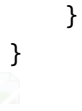
Você foi contratado por uma empresa de desenvolvimento de *software* que está desenvolvendo um *e-commerce*. Sua equipe está desenvolvendo uma classe em Java para calcular o valor total de uma compra realizada pelo cliente.

Seu desafio é analisar o código e escrever casos de teste para cobrir todas as instruções da classe e verificar quantos são necessários para cobrir todas as situações. Você deve considerar as seguintes premissas:

- ◆ O valor da compra é calculado com base na quantidade de itens comprados, no preço unitário de cada item e no desconto que pode ser aplicado.
- ◆ O desconto é determinado com base no valor total da compra. Se o valor em reais for maior ou igual a mil, o desconto é de 10%; se o valor em reais for menor que mil, não há desconto.

O código produzido é o seguinte:

```
public class CalculadoraValorCompra {  
  
    public static double calcular(int quantidade, double precoUnitario, double taxaDesconto) {  
        double total = quantidade * precoUnitario;  
        if (total >= 1000) {  
            total = total - (total * taxaDesconto / 100);  
        }  
        return total;  
    }  
}
```



Encerramento

Neste material, você aprendeu a diferença de testes funcionais e testes não funcionais e verificou conceitos de técnicas de testes de caixa-preta e caixa-branca. Além disso, você foi apresentado a métodos de técnicas de testes e praticou por meio de exemplos.



Desenvolvimento de Sistemas

Ferramentas de testes: *bugtrackers* e automação

Introdução

Como você já sabe, por meio dos testes é possível reduzir a probabilidade de defeitos não descobertos permanecerem no *software*. Para que esses defeitos sejam encontrados de forma antecipada, as atividades de testes devem iniciar o mais cedo possível no ciclo de vida de desenvolvimento de *software*. Além disso, para a realização de testes, existem ferramentas específicas para os tipos de *softwares*/sistemas e contextos. Por meio dessas ferramentas, os testes são executados, os resultados são reportados e as comparações com testes anteriores são realizadas, podendo serem executados os testes repetidamente em qualquer horário do dia.

A seguir, você verá o que são ferramentas de *bugtrackers* e o que são ferramentas de automação.

Ferramentas de *bugtrackers*

As ferramentas de *bugtrackers* são de extrema importância para os testes. Serão abordados, aqui, o conceito de *bugtracker* e as ferramentas que são utilizadas para essa finalidade.

Você sabe o que são *bugtrackers*?

Eles têm como finalidade auxiliar a equipe de teste e de desenvolvedores a manter o histórico de todos os *bugs* que ocorreram no *software*. É nesse tipo de ferramenta que todos os dados referentes aos *bugs* encontrados são colocados. É um meio formal de sinalizar aos desenvolvedores que os *bugs* existem. Ocorre da seguinte forma: os *bugs* encontrados são cadastrados na ferramenta, para que, a partir disso, as equipes possam gerenciá-los.

Ferramentas de automação



A automação de testes está em expansão e, diferente do que os leigos acreditam, não é um processo de testes em si, mas, sim, parte de um processo maior. A implantação de automação geralmente falha quando os testes são imaturos e as responsabilidades não são bem definidas, até mesmo por não terem profissionais suficientemente qualificados ou, ainda, por optarem por ferramentas sem ter a maturidade suficiente para o processo de testes.

O objetivo da automação de testes é a redução do envolvimento humano em atividades repetitivas e manuais, e o grande benefício em automatizar os testes está em aumentar a amplitude e a profundidade da cobertura de testes a serem realizados.

Antes da escolha de uma ferramenta de testes, tanto automatizados quanto manuais, deve-se levar em consideração a criação de uma prova de conceito. Além disso, deve-se ter certeza de que não ocorrem erros críticos e utilizar sempre a versão mais atualizada da ferramenta, realizando teste de regressão. Outros fatores muito importantes são o dimensionamento e a infraestrutura adequados, pois as ferramentas necessitam de computadores com alto desempenho, processadores rápidos e memória grande, os quais devem ser exclusivos para a automação de testes.

Lembrem-se de que a automação não substitui os testes manuais. Ela deve ser um adicional para agregar valor.

Agora, serão apresentadas algumas das ferramentas mais utilizadas em testes, tanto manuais quanto automatizados, ferramentas comerciais e gratuitas.

Ferramentas de *bugtrackers*

Selenium

Para testes automatizados relacionados a aplicações *web* pelo *browser*, há a ferramenta Selenium, utilizada para teste de aceitação, teste funcional, teste de *performance*/desempenho, teste de carga, teste de estresse, teste regressivo, desenvolvimento orientado a testes e desenvolvimento orientado a comportamento. É uma ferramenta gratuita e que fornece recursos de reprodução de gravação. Várias linguagens de programação podem ser escritas por testadores, por exemplo, JavaScript, PHP (*hypertext preprocessor*, ou pré-processador de hipertexto), Python, Ruby, Java, Perl, Groovy e C#.

O *download* da ferramenta pode ser realizado no *site* da Selenium (o qual você encontrar digitando “Selenium” no seu buscador), na aba **Download**. Já a documentação pode ser encontrada no mesmo *site*, clicando em **Documentação – Diretrizes – Tipos de teste**.

A ferramenta é uma extensão do Firefox e depende do *browser* para funcionar. Além disso, também existe disponível o núcleo da Selenium para que seja executado independentemente de plataforma e do Firefox, no qual é disponibilizado somente o JSFTR (JavaScript Functional Test Runner).

Após a instalação do Firefox, o próximo passo é fazer o *download* do Selenium IDE (*integrated development environment*). Para realizar o *download* da extensão, basta clicar em **Extensões** no navegador (ícone em formato de uma peça de quebra-cabeça) e digitar “Selenium” no campo **Encontrar mais extensões**, conforme apresentado na figura 1. Como resultado, será exibida uma tela com as extensões pertinentes ao que foi digitado, conforme apresentado na figura 2.

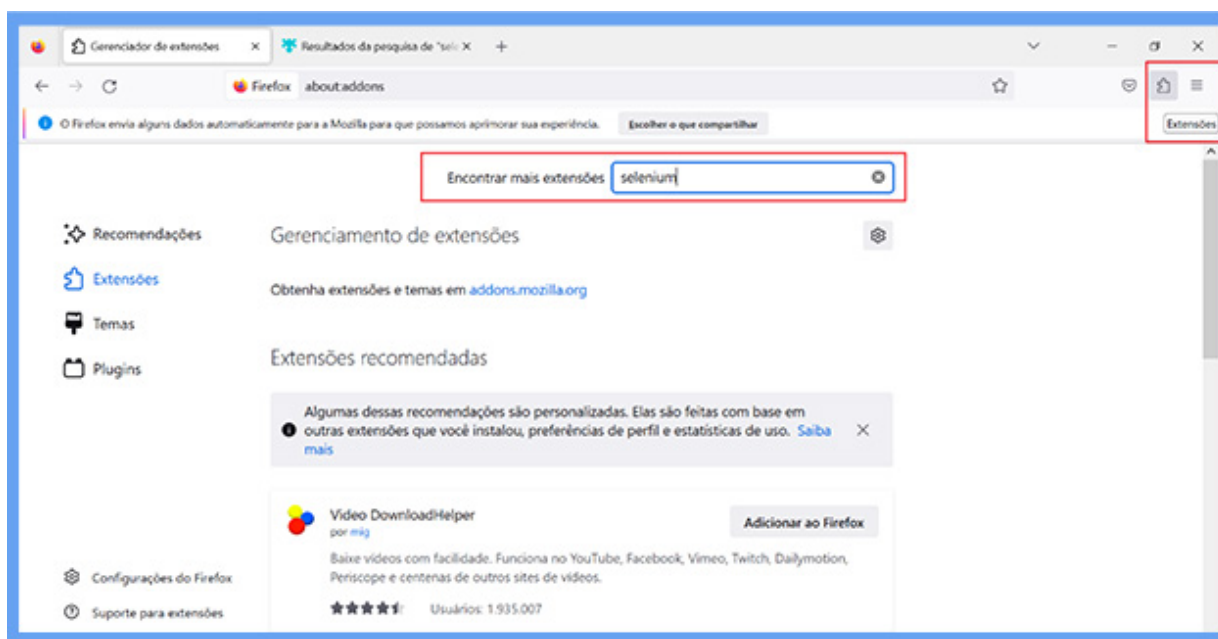


Figura 1 – Exemplo de como encontrar a extensão Selenium IDE no gerenciador de extensões do navegador Firefox

Fonte: Senac EAD (2023)

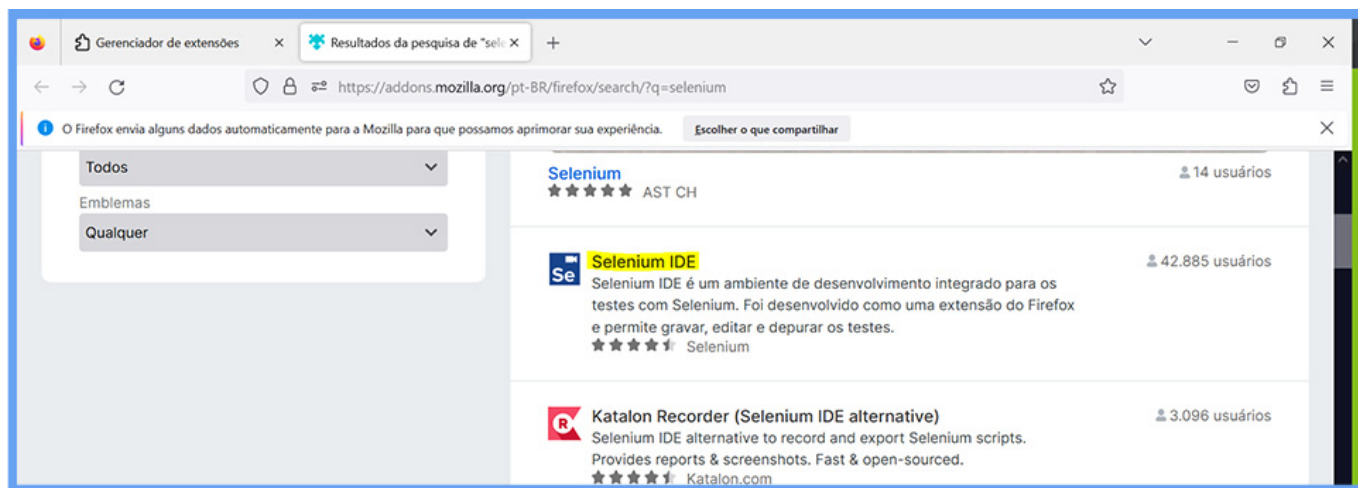


Figura 2 – Extensão Selenium IDE no gerenciador de extensões do navegador Firefox

Fonte: Senac EAD (2023)

A extensão será apresentada para ser realizado o *download* conforme apresentado na figura 3, bastando clicar em **Adicionar ao Firefox**.

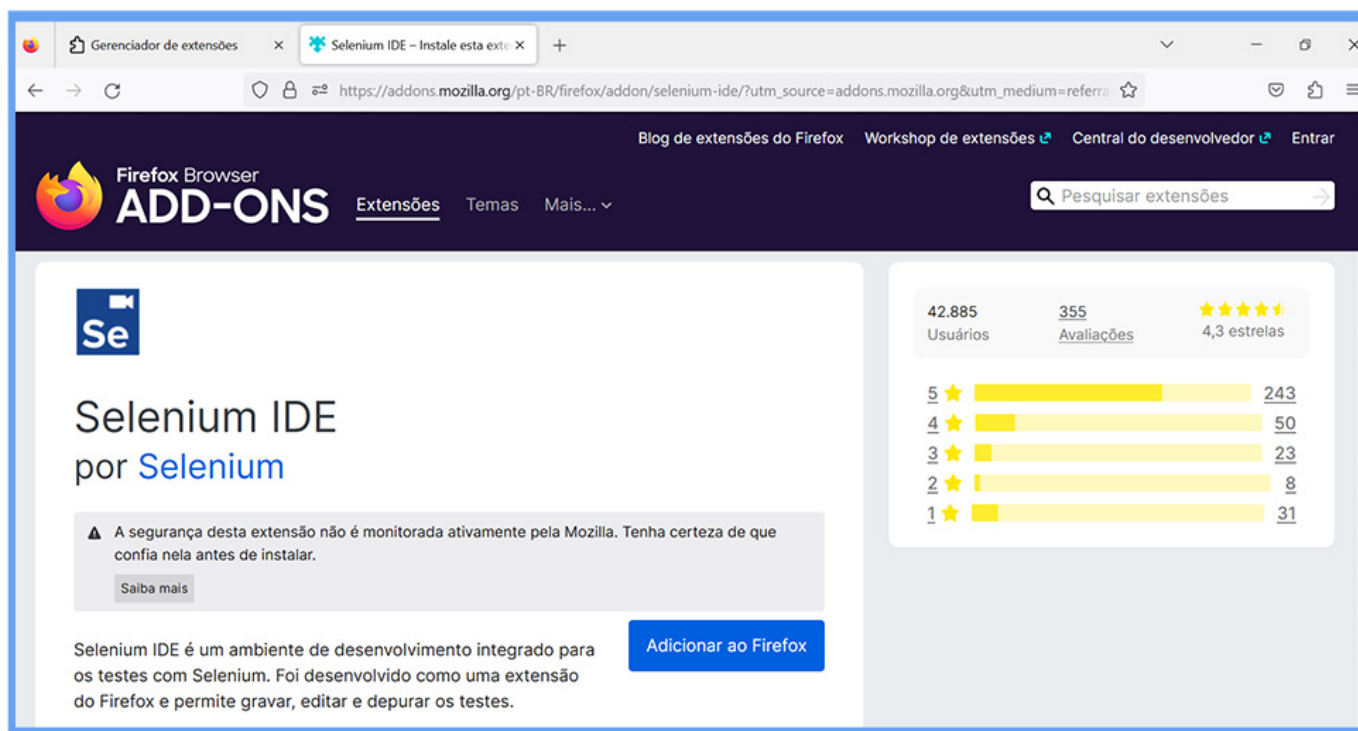


Figura 3 – Extensão Selenium IDE no gerenciador de extensões do navegador Firefox

Fonte: Firefox Browser (2022)

Em seguida, é necessário acessar a extensão por meio do ícone em formato de uma peça de quebra-cabeça e selecionar o que deseja fazer ao executá-la. Assim que estiver instalada, a extensão Selenium IDE está pronta para ser utilizada, assim como é demonstrado na figura 4. Ao selecionar essa opção, será visualizada a interface de configuração dos testes do Selenium IDE, onde é possível optar por gravar um novo teste de um novo projeto, abrir um projeto existente, criar um novo projeto ou simplesmente fechar a extensão.

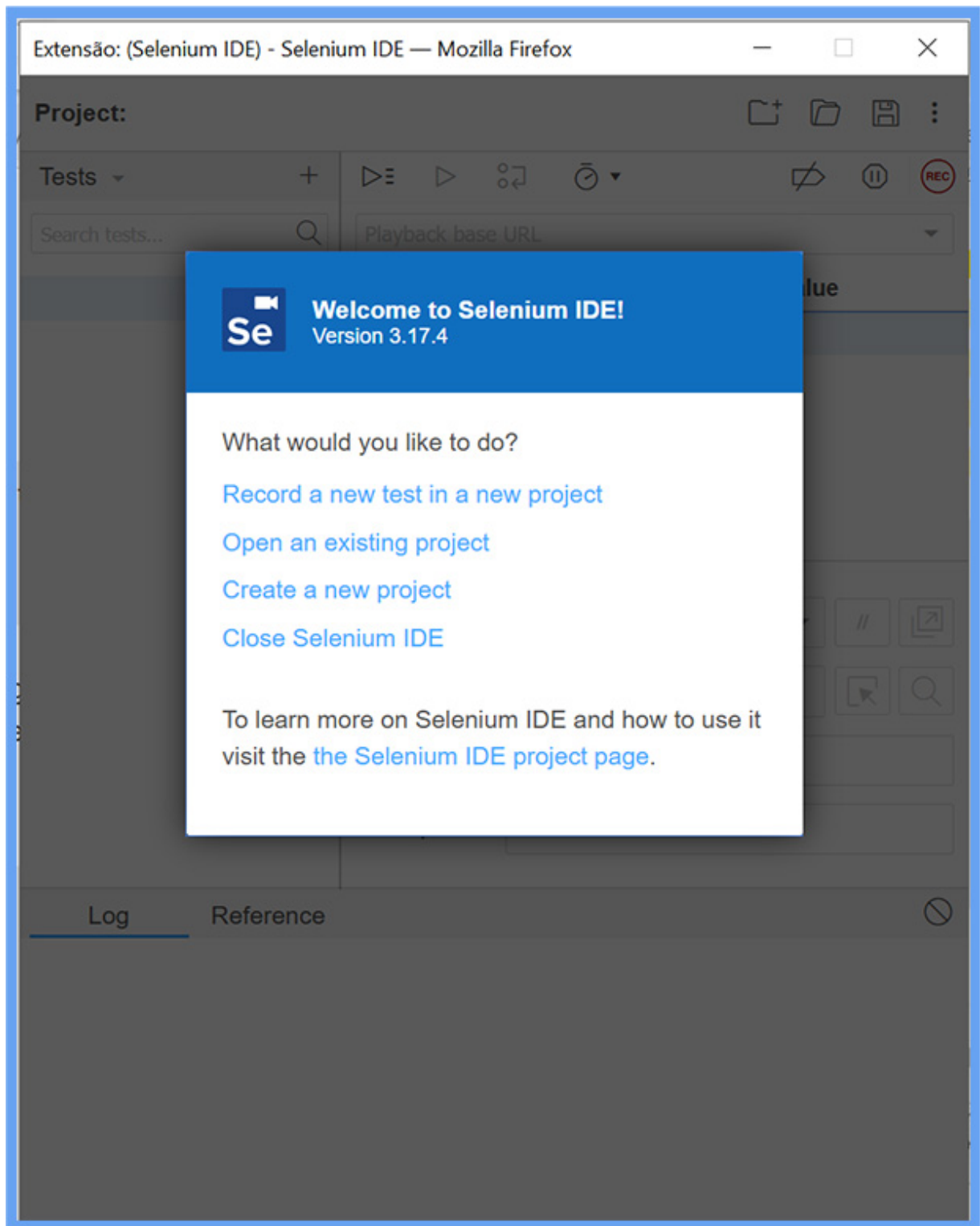


Figura 4 – Exemplo da ferramenta Selenium IDE após clicar para a sua execução

Fonte: Senac EAD (2023)

ZipBoard

O zipBoard é uma ferramenta de rastreamento de *bugs* e *feedback* visual que permite aos desenvolvedores *web* trabalharem de maneira colaborativa em seus produtos e projetos, sejam eles *web apps*, sejam *sites*, entre outros. Essa ferramenta pode ser utilizada durante a criação de

novos *sites*, *e-commerce*, ou ainda para melhorias que sejam pertinentes a produtos já existentes, de forma que auxilie no gerenciamento de tarefas. É uma ferramenta gratuita para utilizar durante 15 dias. Após esse período, torna-se paga.

Algumas de suas características são: utilização *on-line* por meio do navegador sem qualquer necessidade de instalação de *software*; teste de usuário integrado e plataforma de rastreamento de *bugs*; relatórios de erros visuais, nos quais é possível reproduzir *bugs* facilmente e com a utilização de relatórios de *bugs* detalhados dispondo de informações relevantes; medição de desempenho; teste de usuário em que é possível coletar entradas de usuários e clientes sem sair do conteúdo; teste de *design* responsivo incluindo teste de capacidade de resposta do *site*; navegação em diferentes resoluções; capturas de telas; e arquivos *bugs* para resolução específica diretamente do navegador.

A ferramenta pode ser acessada no *site* do zipBoard (digite “zipBoard *bug tracking*” em seu buscador). Além disso, a inscrição é feita no próprio *site* e de forma gratuita, sem a necessidade de adição de cartão de crédito para o período pós-teste.

Na figura 5, é apresentado o *layout* da ferramenta, em que é possível cadastrar novos projetos e novas tarefas, deixar mensagens e verificar reportes. Também há a possibilidade de realizar integrações com outras plataformas, por exemplo, Jira, Slack, Microsoft Teams e Wrike para sincronização automatizada.

Figura 5 – Exemplo da ferramenta zipBoard

Fonte: Senac EAD (2023)

Jira Software

Jira é a principal ferramenta de rastreamento de *bugs*, rastreamento de problemas e gerenciamento de projetos. O Jira combina rastreamento de problemas, gerenciamento de projetos ágil, fluxo de trabalho personalizável e uma estrutura de integração por meio de *plugins* para aumentar a velocidade de sua equipe de desenvolvimento de *software*. Além disso, o Jira é gratuito para equipes de até 10 pessoas e centraliza relatórios de *bugs*, com a possibilidade de adicionar contexto a cada tarefa com rótulos, campos personalizados, anexos, entre outros.

Há a possibilidade de eliminar os *bugs* antes que problemas maiores ocorram, priorizando e atribuindo tarefas de acordo com a urgência em que se apresentam.

Ainda sobre as questões de *bugs*, é possível utilizar um fluxo de trabalho criado especificamente para a resolução deles, mantendo o time com foco no que for preciso. Na figura 6, é apresentado um exemplo de um quadro em que constam os *bugs* mapeados (todos os que têm o ícone quadrado vermelho com um círculo branco ao centro).

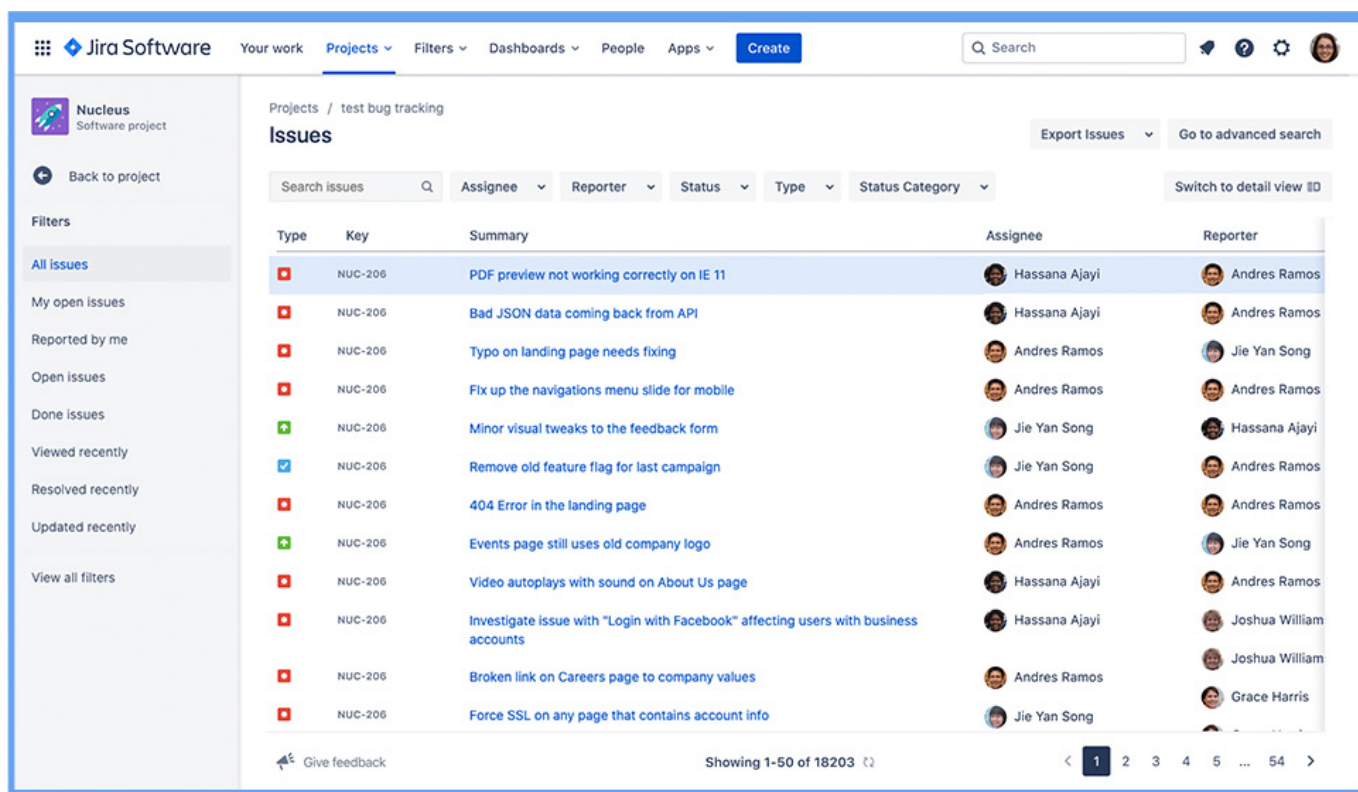


Figura 6 – Exemplo da ferramenta Jira

Fonte: Atlassian (c2023b)

Para iniciar os trabalhos no Jira, é necessária a criação da conta na plataforma. Ao acessar o endereço, aparecerá a tela apresentada na figura 7, na qual se pode escolher se a preferência é por criar uma conta com um *e-mail* ou, ainda, utilizar o Google.

Figura 7 – Exemplo da ferramenta Jira

Fonte: Atlassian (c2023c)

Após o cadastro (você precisa optar por uma das duas formas), aparecerá a tela de boas-vindas, na qual você disponibilizará o nome de seu *site*, conforme apresentado na figura 8 a seguir.

Figura 8 – Exemplo da ferramenta Jira

Fonte: Senac EAD (2023)

Após inserir as informações, a plataforma inicializará seu *board* e você precisará aguardar alguns minutos, conforme apresentado na figura 9.

Figura 9 – Tela de criação do *board*

Fonte: Senac EAD (2023)

Os próximos passos são: adicionar as informações referentes ao projeto no qual está sendo utilizado o *template* de controle de *bugs* escolhido por você anteriormente, criar a chave que identificará o projeto e, finalmente, clicar em **Criar projeto**, conforme apresentado na figura 10.

Figura 10 – *Template* de adição de informações e finalização da criação do projeto

Fonte: Senac EAD (2023)

Há, ainda, a possibilidade de utilizar outras ferramentas que serão integradas e conectadas posteriormente, por exemplo, Slack (mensageiro), Microsoft Teams (mensageiro), GitHub (repositório de códigos), GitLab (repositório de códigos), Jenkins, entre outros.

Pronto! Agora você pode utilizar o *board* criado para o seu projeto. Para criar o primeiro item, basta clicar em **Criar** no menu superior, conforme demonstrado na figura 11.

Figura 11 – *Board* onde ficarão os itens que serão criados no decorrer do projeto

Fonte: Senac EAD (2023)

Por meio da ferramenta, você pode capturar *bugs* em qualquer lugar em seus projetos de *software* com o Jira Software. É possível identificar um *bug*, criar um problema e adicionar todos os detalhes relevantes, incluindo descrições, nível de gravidade, capturas de tela, versão e muito mais, conforme apresentado na figura 12.

Os problemas podem representar qualquer coisa, desde um *bug* de *software* ou uma tarefa de projeto até um formulário de solicitação de licença, e cada tipo de problema exclusivo pode ter o próprio fluxo de trabalho personalizado.

Figura 12 – Tela de cadastro de *bug*

Fonte: Senac EAD (2023)

Os campos obrigatórios devem ser preenchidos, assim como os demais campos pertinentes ao projeto. Após o preenchimento, basta clicar em **Criar** e o item terá sido criado no *board* para ser acompanhado, conforme mostra a figura 13.

Figura 13 – Tela do *board* com o item cadastrado



Fonte: Senac EAD (2023)

Quando os *bugs* são capturados, você pode priorizá-los de acordo com o nível de importância, urgência e capacidade da equipe, atribuindo-os com alguns cliques e priorizando-os por meio do “arrasta e solta” na lista de pendências ou, ainda, na coluna de tarefas pendentes da equipe, conforme apresentado na figura 14.

Figura 14 – Exemplo de *backlog* de tarefas que o time tem para realizar

Fonte: Atlassian (c2023a)

Após a criação e a atribuição do item, é possível realizar a alteração de seu *status* na página inicial do *board*. Clicando em **Status**, aparecem as opções, conforme a figura 15.

Figura 15 – Exemplo de alteração de *status*

Fonte: Senac EAD (2023)

Há também como estar sempre informado rastreando *bugs* e problemas por meio do fluxo de trabalho de sua equipe. O Jira Software tem um poderoso mecanismo de fluxo de trabalho, recursos de planejamento, pesquisa abrangente e recursos de geração de relatórios projetados para ajudar a encontrar, registrar e rastrear *bugs* em seu *software*, conforme o exemplo mostrado na figura 16.

Figura 16 – Exemplo de fluxo de trabalho de equipe

Fonte: Site da Atlassian (c2023a)

Além disso, é possível manter-se atualizado com as notificações, pois não são somente as menções, e também configurar para que os responsáveis sejam notificados de forma automática a cada mudança de *status*, mantendo, assim, a visibilidade e a transparência entre todos os membros.

Agora que você já aprendeu sobre *bugtrackers* e ferramentas que exercem essa função, você aprenderá sobre as ferramentas de automação.

Ferramentas de teste: automação



Há também a possibilidade de utilizar o Jira para a realização dos testes. O Jira é uma ferramenta de *software cloud* desenvolvida pela australiana Atlassian que permite monitorar tarefas e projetos de forma que seja garantido o gerenciamento em um local único. No Jira, há extensões que podem ser instaladas a fim de permitir que se façam inúmeras atividades, dentre elas a questão de testes, sejam manuais, sejam automatizados.

Uma dessas extensões a baixo custo após o período de experimentação (menos de 1 dólar por integrante de equipe para até dez pessoas) é a Xray. Essa extensão tem como objetivo o gerenciamento de teste nativo, ou seja, permite planejar, projetar e executar testes, assim como gerar relatórios deles.

Além disso, para cada fase de teste, é permitido que sejam utilizados os seguintes itens:

Fase do plano, em que constam os itens de plano de teste

Fase de projeto, em que a especificação é definida por meio de tipos de itens de pré-condição e teste

Fase de execução, em que estarão os itens de execução de teste

Fase de relatório, em que estão inclusos os itens de execução de teste, relatórios de cobertura de requisitos integrados e *tickets* utilizando as ferramentas do Jira

Será abordado, aqui, um passo a passo para a utilização do Xray, iniciando em como fazer a instalação. Para instalar, é necessário ter uma conta criada no Jira e procurar, na página do Atlassian Marketplace (digite “Atlassian Marketplace” no seu buscador), por “Xray Test Management for Jira”, selecionando a opção de fazer teste gratuito.

Agora que você já está com a extensão instalada, veja como escrever e especificar testes. É preciso saber que há três tipos de teste padrão: **pepino**, em que se pode escrever testes em um domínio legível para negócios e linguagem específica (Gherkin); **genérico**, em que há um teste não estruturado, sem etapas, sendo uma maneira de abstrair e ter a visibilidade de testes automatizados ou exploratórios; e **manual**.

Para criar um novo item de teste, clique em **Criar item** no menu superior da tela, conforme a figura 17 apresentada a seguir.

Figura 17 – Exemplo do menu de criação de item de teste

Fonte: Freire (c2023)

A próxima ação é selecionar o projeto no **Tipo de item** e selecionar **Testar**. Em seguida, digite a descrição resumida para o teste. É necessário preencher todos os campos obrigatórios que estão sinalizados com o asterisco e, na sequência, clicar em **Create**, se estiver tudo de acordo com o esperado, conforme apresentado na figura 18.

Figura 18 – Exemplo do menu de criação de item de teste

Fonte: Freire (c2023)

Após completar essa primeira etapa, será abordado como organizar os testes no Xray. Essa é uma etapa de grande importância para as atividades de teste diárias. O Xray oferece duas formas: a primeira é usar os itens de conjuntos de testes, já a segunda é utilizar o repositório de teste. Um conjunto de testes é uma forma de criar diversos grupos de testes, e um teste pode ser incluído em vários conjuntos. Já um repositório de testes é uma estrutura organizada em árvore no nível do projeto, o que torna possível que os testes sejam organizados por hierarquia nas pastas e nas subpastas.

Para criar um novo item de conjunto de testes, é necessário clicar em **Criar item**, na parte superior da tela, para que abra a referida página. Em seguida, selecione o “projeto” e vá em **Tipo de item, Conjunto de testes**. O próximo passo é digitar um resumo para o teste e preencher os campos obrigatórios. Realizado esse passo, basta clicar em **Criar**.

Para a criação de um repositório de teste, é necessária a criação de uma nova pasta.

Esse procedimento é feito da seguinte forma:

Clicar na pasta.

Clicar no ícone de adicionar (sinal de +), na parte superior da seção de pastas, conforme apresentado na figura 19.

Figura 19 – Exemplo de criação de repositório de teste

Fonte: Freire (c2023)

Digitar o nome desejado para a pasta na caixa de diálogo de criação de pasta, conforme apresentado na figura 20.



Por fim, clicar em **Create** para criar a pasta.

Figura 20 – Exemplo de criação de repositório de teste

Fonte: Freire (c2023)

Conforme visto até o momento, você já sabe como criar um conjunto de testes e como criar um repositório de testes. O próximo passo será verificar como planejar testes. Mas você sabe para que serve o planejamento de testes? O planejamento de testes consente que seja decidida por você a estratégia de teste, em que estão inclusos os itens que você quer validar, a forma de fazer essa validação, se serão testes manuais ou automatizados, de que maneira os recursos serão alocados e o momento e por quem os testes serão executados. Dessa forma, é possível a priorização de testes ou a concentração em requisitos específicos. E de que forma é possível fazer isso no Xray no Jira? Siga em frente para aprender!

Realize o passo a passo para a criação do plano de teste:

Primeiro, é necessário clicar em **Create**, na parte superior da tela, para abrir a página **Criar item**, conforme apresentado na figura 21.

Figura 21 – Exemplo de criação de plano de teste.

Fonte: Freire (c2023)

A etapa seguinte consiste em selecionar o projeto e, em **Tipo de item**, selecionar **Plano de teste**.

Da mesma forma como ocorreram em etapas de processos anteriores, é necessário preencher um resumo e todos os campos obrigatórios presentes na tela e, finalmente, clicar no botão **Create**, conforme apresentado na figura 22.

Figura 22 – Exemplo de criação de plano de teste

Fonte: Freire (c2023)

Por fim, há a etapa de execução dos testes. A execução dos testes acontece entre o desenvolvimento de *scripts* e relatórios e a análise de resultados do teste. Essa execução é um tipo de item que acrescenta uma coleção de testes escolhida pelo usuário, em que ele monitora e verifica se os testes estão ocorrendo de acordo com o esperado no contexto e no ambiente destinado. Agora que você já sabe quando ocorre a execução de testes, veja como criar um item de execução de teste:

Clicando no sinal de soma apresentado no canto superior esquerdo da página, abrirá a página de criação de item, conforme pode ser visto na figura 23.



Figura 23 – Exemplo de página onde inicia a criação de um item de execução de teste

Fonte: Freire (c2023)

Diante do que foi apresentado, pode-se concluir que o Xray é uma boa opção para utilizar no Jira, dispondo de vários recursos e sendo uma ferramenta intuitiva para o usuário.

Há outras ferramentas que podem ser utilizadas para a gestão de testes, algumas delas comerciais (pagas), como Mercury Interactive Test Director, Compuware QA Director e IBM Rational Test Manager, e outras *open source* (gratuitas), como Bugzilla Test Runner, Software Testing Automation Framework (STAF), JWebUnit, Bugzilla (testes funcionais), JMeter (*performance*), Mantis e phpBugTracker (sistemas *bugtracking*).

A ferramenta Bugzilla é um sistema de gestão de casos de teste que permite aos programadores a manutenção e a gestão de *bugs* do *software* em questão. Além disso, permite registrar alterações de código e *bugs* (problemas), comunicação entre membros da equipe, submissão e revisão de pacotes e gestão de QA (*quality assurance*). Na figura 24, é possível visualizar a interface na aplicação.

Figura 24 – Interface do Bugzilla

Fonte: Bugzilla (2023)

A ferramenta Mercury Interactive – Quality Center é uma suíte de aplicações para testes de *software* que permite gerir e automatizar o processo de QA e tem central de testes baseada na *web*. Sua ferramenta de gestão de testes é o TestDirector. Além disso, dentro do processo de teste, há grande destaque para metodologia, estrutura, organização e documentação.

O TestDirector é utilizado para identificar e definir os requisitos funcionais e de *performance* de teste e para realizar os testes. Também é possível desenvolver o plano de testes, executar os testes e registrar os defeitos encontrados e sua correção. Por meio da ferramenta, definem-se requisitos e propriedades, árvores de requisitos, agrupamento de requisitos por grupos funcionais

e tipos, bem como seus estados, alterações de requisitos com seu respectivo histórico, anexação de documentos nos requisitos, envio de dados por *e-mail* e três visões da gestão de requisitos, sendo elas, respectivamente: lista de requisitos e propriedades, testes de cada requisito e execuções de teste de cada requisito.

Já os planos de teste precisam ser elaborados e organizados de acordo com as unidades funcionais da aplicação que serão testadas. Ainda, é preciso priorizar as relações hierárquicas e de dependência entre testes.

A seguir, na figura 25, apresenta-se a interface da ferramenta. É importante salientar que é uma ferramenta de mercado, portanto não é gratuita, e pode ser acessada digitando “TestDirect” em seu buscador.

Figura 25 – Interface do *dashboard* do TestDirector

Fonte: TestDirector (2023)

A Mercury afirma que essa é uma ferramenta líder de mercado. Além de ser uma ferramenta completa e em nuvem, há a opção de instalação no próprio servidor da empresa, em que se deve ter ao menos dez usuários, optar pelo plano *enterprise* e cumprir uma série de requisitos de infraestrutura necessários.

Encerramento

Espera-se que este conteúdo tenha conseguido contribuir para o seu conhecimento acerca de ferramentas de testes manuais e automatizados. Aqui, você pôde conhecer as principais ferramentas, onde encontrá-las e a forma de utilizá-las. Diante do que foi abordado, agora você pode optar por ferramentas de mercado ou gratuitas e ainda entender como aplicá-las no dia a dia.



Desenvolvimento de Sistemas

Ferramentas de depuração passo a passo

No contexto de testes, ferramentas que permitam a análise da execução de um código são bem-vindas, especialmente como testes de caixa-branca. Elas permitem a análise do que está acontecendo no código em tempo real.

O processo de depuração complementa o processo de testes. Enquanto o teste identifica os erros que acontecem no programa e os efeitos de uma falha, a depuração (ou *debugging*) serve para localizar onde o erro se origina e planejar como corrigi-lo.

Felizmente, os IDEs (*integrated development environment*, ou ambiente de desenvolvimento integrado), desde 1983, com o lançamento do Borland Turbo Pascal, trazem ferramentas que permitem, baseando-se no ambiente de desenvolvimento, um processo de depuração, incluindo pontos de parada no código, executando passo a passos as instruções programadas e permitindo a visualização dos valores guardados em memória. Esses recursos estão presentes e aprimorados em IDEs modernos, como o NetBeans.

A seguir, veja os comandos presentes para a depuração no NetBeans e suas utilizações.

Recursos de IDE

De maneira sucinta, é assim que se realiza a depuração automatizada com NetBeans IDE:

1. Marcar um *breakpoint* (ponto de parada) na linha de código desejada. Para fazer isso, deve-se clicar, na barra lateral onde ficam os números das linhas, na linha em que se deseja que a execução pause (ou seja, o programa executará todas as instruções que acontecerem antes da marcada com *breakpoint*).

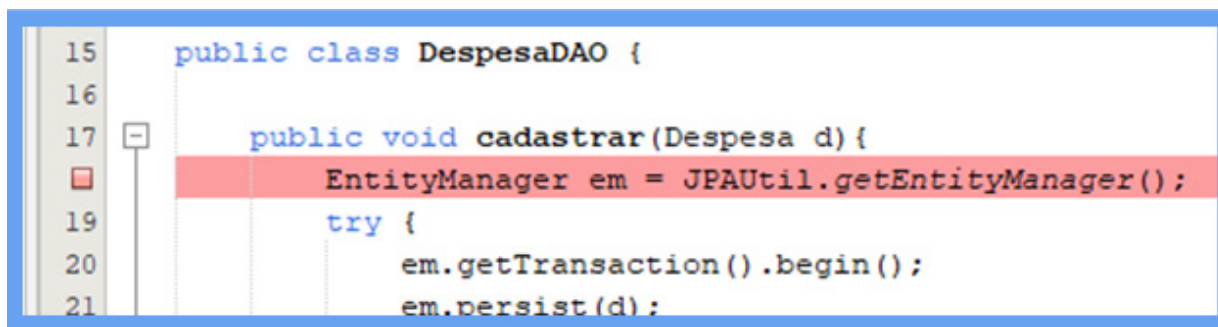


Figura 1 – *Breakpoint* posicionado na linha 18 do código

Fonte: Senac EAD (2023)

2. Acionar a depuração pelo ícone **Debug Project** na barra principal de ferramentas.

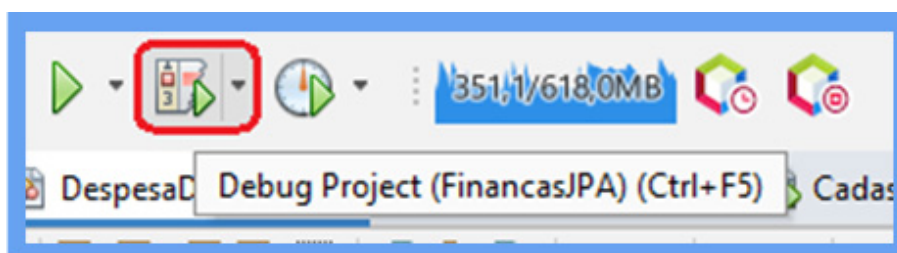


Figura 2 – Ícone **Debug Project**

Fonte: Senac EAD (2023)

3. Acionar a barra de ferramentas de *debug*, o que acontece quando se aciona o *debug*. Em breve, cada um desses recursos será lembrado.

Figura 3 – Ferramentas de *debug* do NetBeans

Fonte: Senac EAD (2023)

4. Avançar linha a linha por meio das ferramentas de *debug*.

No momento em que está pausada a execução em um *breakpoint*, é possível utilizar um dos recursos essenciais da depuração: a visualização dos valores em memória. Para isso, pode-se passar com o *mouse* sobre o nome de uma variável ou um objeto, e um painel *pop-up* abrirá com os valores.

Figura 4 – Visualização de valores no editor do NetBeans. Abaixo, a visão expandida, mostrando os valores dos atributos do objeto **novaDespesa**

Fonte: Senac EAD (2023)

As variáveis podem ser observadas também no painel **Variables**, no qual é possível cadastrar um observador (ou *watch*) para uma variável ou um objeto. Por padrão, o painel, que fica na parte inferior da janela do NetBeans, mostra as variáveis presentes no escopo atual da execução.

Figura 5 – Aba **Variables** no NetBeans

Fonte: Senac EAD (2023)

Contudo, também é possível incluir um novo *watch*, ou seja, incluir um monitoramento para alguma variável, algum atributo ou um objeto qualquer. No campo **<Enter new watch>**, basta incluir a expressão que se quer monitorar. Isso também pode ser feito por meio do ícone de diamante com um sinal de **+** (**Create new watch**).

Figura 6 – Incluindo novo *watch* para a expressão **txtDescricao.getText()**

Fonte: Senac EAD (2023)

A expressão não precisa ser necessariamente um nome de variável. Pode ser a invocação de um método, por exemplo.

Note, ainda, que na região inferior da tela há uma aba **Breakpoints** em que se pode monitorar todos os pontos de parada incluídos no projeto.

Figura 7 – Aba **Breakpoints** no NetBeans

Fonte: Senac EAD (2023)

Comandos e utilização

A tabela a seguir resume os comandos presentes na barra de ferramentas de depuração e quando eles devem ser usados.

Botão	Descrição	Quando usar
Finish Debugger Session	Encerra a sessão de depuração	Ao encontrar uma falha, permite continuar a execução até o fim ou simplesmente encerrar o programa usando esse botão.
Pause	Pausa a execução	Se um programa está executando, mas ainda não atingiu um ponto de parada, esse recurso pode ser usado para pausar sua execução.
Continue	Segue a execução	Usado para, caso seja necessário, seguir a execução sem ir passo a passo até atingir outro <i>breakpoint</i> posterior ou o fim do programa.
Step Over	Pular para a próxima linha	Segue a execução para a linha imediatamente posterior à atual.
Step Over Expression	Pular expressão	Executa expressão a expressão de uma linha. Por exemplo: em.getTransaction().begin(); O comando passaria primeiro pelo getTransaction() e depois pelo begin() .
Step Into	Entrar na execução de uma expressão	Executa o passo a passo de um método invocado pela linha atualmente em execução.
Step Out	Sair da execução de uma expressão	Segue com a execução de um método sem ir passo a passo e retornar à linha onde esse método foi invocado.
Run to Cursor	Pular para o cursor	Sem necessitar criar um novo <i>breakpoint</i> , pula a execução direto para a linha onde o cursor está posicionado.
Apply Code Changes	Aplicar mudanças em código	Normalmente, é preciso encerrar a depuração para fazer uma alteração no código e depois reexecutar o <i>debug</i> para usar as alterações executadas. Esse recurso aplica as alterações de código sem parar a execução do <i>debug</i> .
Toggle Pause in GraalVM Script	Pausar GraalVM	Usado para linguagens Ruby e JavaScript.
Take GUI Snapshot	<i>Print</i> da tela do programa	Tira um <i>print</i> da tela em execução no projeto (JFrame).

Tabela 1 – Comandos presentes na barra de ferramentas de depuração

Fonte: Senac EAD (2023)

Encerramento

O recurso de depuração, presente hoje na maioria dos IDEs, é indispensável na rotina de um programador, pois ajuda a localizar exatamente o que causa uma falha no programa.

As ferramentas disponíveis no NetBeans são bastante abrangentes e permitem um processo bem detalhado de depuração, ajudando o desenvolvedor nessa missão.